SIMATS ENGINEERING

ANALYTICAL PROBLEM SOLVING

Questions and Answers

1. Water Jug Problem

Question: Two unmarked jugs have capacities of 4 gallons and 3 gallons. Using a pump, the ground, and transfers between jugs, obtain exactly 2 gallons in the 4-gallon jug.

Answer: Represent each state as (x, y), where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug.

Step	Action	State (4-gal, 3-gal)
0	Both jugs start empty.	(0, 0)
1	Fill the 3-gallon jug.	(0, 3)
2	Pour the 3-gallon jug into the 4-gallon jug.	(3, 0)
3	Fill the 3-gallon jug again.	(3, 3)
4	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. Only 1 gallon transfers; 2 gallons remain in the 3-gallon jug.	(4, 2)
5	Empty the 4-gallon jug onto the ground.	(0, 2)
6	Pour the remaining 2 gallons into the 4-gallon jug.	**(2, 0)**

Result: The 4-gallon jug contains exactly 2 gallons.

2. Mars Rover: Intelligent Agent Analysis

i. Percepts: Camera images, distance and proximity readings, wheel/position sensors, terrain slope, temperature, radiation, atmospheric readings, and data from scientific instruments that analyse rock, soil, and other samples.

ii. Operating environment:

• **Partially observable:** The rover cannot directly observe every feature or hazard around it.

• **Stochastic:** Terrain, dust, wheel slip, and equipment behaviour can make outcomes uncertain.

• **Sequential:** Each movement or sample test affects later choices.

• **Dynamic:** Lighting, temperature, and environmental conditions can change.

• **Continuous and partly unknown:** Position, movement, and sensor values vary continuously, and the Martian environment is not fully known in advance.

• **Single-agent:** The rover performs the mission independently, although Earth-based operators may provide goals and instructions.

iii. Actions: Move forward/backward, turn, stop, capture images, navigate around obstacles, deploy an arm or drill, collect and analyse samples, transmit data, and enter a safe mode or recharge where supported.

iv. Performance evaluation: Measure scientific value and accuracy of collected data, number and quality of samples analysed, distance and area explored, mission objectives completed, energy and time used, communication success, and safety/reliability (avoiding damage and hazards).

v. Suitable architecture: A **model-based, goal-based agent with utility-based decision-making** is most suitable. It maintains an internal map/state because the world is only partly observable, plans actions to reach scientific goals, and uses utility estimates to balance scientific benefit, energy, time, risk, and equipment safety. Learning from previous observations can improve future navigation and sample selection.

3. Eight-Queens Problem: Formulation and Search

Problem: Place eight queens on an 8 × 8 chessboard so that no two queens share a row, column, or diagonal.

Problem components:

• **Initial state:** An empty chessboard.

• **State representation:** A sequence [r■, r■, …, r■], where r■ is the row of the queen in column i; k queens have been placed in the first k columns.

• **Actions:** Place a queen in any safe row of the next column.

• **Transition model:** Add a queen in the next column if it does not conflict with existing queens.

• **Goal test:** Eight queens are placed and no pair shares a row, column, or diagonal.

• **Path cost:** Usually 1 per placement; all complete solutions have the same cost of eight placements.

Search strategy: Depth-first search with backtracking is effective. Place one queen per column, try safe rows, and backtrack whenever no safe row remains. Constraint-satisfaction techniques can further reduce the search by checking row and diagonal conflicts immediately.

Example valid solution: Row positions by columns [1, 5, 8, 6, 3, 7, 2, 4]. This is a solution because each row is unique and no two queens lie on the same diagonal. The configuration described in the question is not a solution if the first- and last-column queens share a diagonal.

Backtracking pseudocode:

```
BACKTRACK(column):
if column == 9:
report the arrangement
return
for row = 1 to 8:
if row and both diagonals are safe:
place queen at (row, column)
BACKTRACK(column + 1)
remove queen at (row, column)
```

4. OLA Cab Booking: Problem-Solving Agent

Suitable agent: A **utility-based goal-oriented problem-solving agent**. The goal is to reach the destination, while utility helps compare Mini, Micro, Sedan, Shared, Prime, and other available options based on fare, estimated arrival time, comfort, availability, and the customer's preferences.

Problem formulation:

• **Initial state:** Customer's current location and destination are entered.

• **Actions:** Select a cab category, request a ride, and confirm a suitable available cab.

• **Goal state:** A cab is booked and the customer reaches the destination.

• **Constraints:** Budget, passenger count, comfort preference, pickup time, and cab availability.

• **Performance measure:** Successful booking and arrival, with suitable cost, waiting time, and comfort.

Pseudocode:

```
INPUT pickup, destination, budget, comfort, passengers
available_cabs = get_available_cabs(pickup, destination)

FOR each cab IN available_cabs:
estimate fare, pickup_time, trip_time
check passenger_capacity
score cab using:
budget fit, comfort fit, availability,
estimated waiting time, and trip time

REMOVE cabs that violate essential constraints

IF no cabs remain:
DISPLAY "No suitable cab available; change preferences"
ELSE:
SHOW cabs ranked by score and estimated fare
customer selects a cab
IF customer confirms:
BOOK selected cab
DISPLAY booking details and driver information
```

Note: The final choice should remain with the customer; the agent recommends and ranks options rather than booking without confirmation.

5. Uniform Cost Search (UCS): Least-Cost Delivery Route

Question: Find the least-cost path from start node S to goal node G in the weighted delivery network using Uniform Cost Search. Edge labels represent travel cost.

Graph edges read from the supplied figure: S→A (1), S→G (12), A→B (3), A→C (1), B→D (3), C→D (1), C→G (2), and D→G (3).

UCS principle: UCS expands the frontier path with the lowest cumulative cost g(n). It uses a priority queue and stops when G is removed from the queue as the lowest-cost frontier node—not merely when G is first generated.

Search steps:

Expanded node	Candidate paths / cumulative costs
S (0)	A: 1; G: 12
A (1)	C via A: 2; B via A: 4; G: 12
C (2)	D via C: 3; G via C: 4
D (3)	G via D: 6 (worse than the existing cost 4)
B (4) / G (4)	B can produce D at cost 7, which is worse. G is then removed from the frontier at cost 4, so the search stops.

Least-cost path: S → A → C → G

Total cost: 1 + 1 + 2 = **4**. Other example routes include S→A→C→D→G with cost 1 + 1 + 1 + 3 = 6, and direct S→G with cost 12. Therefore, UCS returns S→A→C→G with minimum total cost 4.

UCS pseudocode:

```
UCS(start, goal):
frontier = priority queue containing (cost=0, start)
best_cost[start] = 0
parent[start] = NONE

while frontier is not empty:
(cost, node) = remove lowest-cost entry
if cost != best_cost[node]:
continue
if node == goal:
return reconstruct_path(parent, goal), cost

for each edge node -> next with edge_cost:
new_cost = cost + edge_cost
if next not in best_cost OR new_cost < best_cost[next]:
best_cost[next] = new_cost
parent[next] = node
insert (new_cost, next) into frontier

return "No path found
```

Conclusion: These problems illustrate state-space search, agent design, constraint satisfaction, and cost-based path finding. Correct state representation and a suitable search strategy help an agent reach its goal efficiently.